

UNIVERSITAT POLITÈCNICA DE CATALUNYA

School of Professional & Executive Development

Postgrado en

Mobile Apps Design

ALARMS

Juan Luis Gorricho

SCHEDULING TASKS

- The alarms, using **AlarmManager**, allow the execution of tasks scheduled on time outside the cycle of life of an app. A typical example would be to synchronize data from a remote server.
- It is recommended to program the alarms:
  - with the lowest frequency,
  - of the appropriate type, that is, use inexact repetition if possible, so **Android** can execute jointly several tasks thanks to its temporal vicinity,
  - setting temporal intervals for repetition instead of exact clock times.

ALARMS

- The programming of alarms is useful to execute a service, to send a broadcast message or even to launch an activity without the user intervention.
- The scheduling of alarms implies launching a future PendingIntent, it can be:
  - **Only one time**: at a given clock time or after a given time interval.
  - **Periodically**: with or without exact repetition, starting at a given clock time or time interval, and finally, every given time interval.
  - The alarm can be **of type**: execute anyway, that is, wake up the phone if necessary, or do not execute if it is in standby.

ALARMS

- Alarm types:
  - AlarmManager.RTC
  - AlarmManager.RTC_WAKEUP
  - AlarmManager.ELAPSED_REALTIME
  - AlarmManager.ELAPSED_REALTIME_WAKEUP

- Examples for scheduling alarms:

```
alarmMgr.set(RTC, System.currentTimeMillis()+10000, pendingIntent);

alarmMgr.setInexactRepeating(ELAPSED_REALTIME,
SystemClock.elapsedRealtime() + INTERVAL_FIFTEEN_MINUTES,
INTERVAL_FIFTEEN_MINUTES, pendingIntent);
```

PENDING INTENT

- a PendingIntent is a wrapper of an **Intent** to be issued in the future.
- Instances of this Java class are obtained from: **getActivity**(Context, int, Intent, int), **getBroadcast**(Context, int, Intent, int), and **getService**(Context, int, Intent, int);
- the returned PendingIntent can be handed over to other applications so that they can perform the embedded intent **on your behalf** at a later time.
- By giving a PendingIntent to another application, you are **granting** it the right to perform the operation you have specified as if the other application was yourself (with the same permissions and identity).